



Cheatsheet de Programación ESP32

Guía rápida de instrucciones y funciones para la asignatura de Informática Industrial

Por [Juan M. Gandarias](#) | jmgandarias@uma.es

0 Configuración Inicial y Setup

Estructura Básica de un Programa

```
const int LED_PIN = 2;

void setup() {
  Serial.begin(115200);
  delay(1000);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("ESP32 iniciado");
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(1000);
  digitalWrite(LED_PIN, LOW);
  delay(1000);
}
```



Comunicación Serial (Debug)

Función	Descripción
<code>Serial.begin(baudrate)</code>	Inicializa puerto serie (115200 recomendado)
<code>Serial.print()</code>	Imprime sin salto de línea
<code>Serial.println()</code>	Imprime con salto de línea
<code>Serial.printf()</code>	Imprime con formato
<code>Serial.available()</code>	Bytes disponibles en buffer
<code>Serial.read()</code>	Lee un byte
<code>Serial.readString()</code>	Lee línea completa

1 Entrada y Salida Digital, Analógica y PWM (LEDC)



GPIOs y Lectura Analógica

Función	Descripción
<code>pinMode(pin, mode)</code>	Configura un pin como INPUT , OUTPUT o INPUT_PULLUP (activa resistencia interna pull-up)
<code>digitalWrite(pin, val)</code>	Establece estado de salida digital (HIGH o LOW)
<code>digitalRead(pin)</code>	Lee el estado lógico de un pin → HIGH (1) o LOW (0)
<code>analogRead(pin)</code>	Conversión A/D → Devuelve entero entre 0-4095 (12 bits por defecto)

Ejemplo:

```
const int BUTTON = 4;
const int LED = 2;

void setup() {
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
}
```

⚡ Control PWM con LEDC

La gestión PWM con LEDC simplifica la configuración: frecuencia y resolución se enlazan directamente al pin sin necesidad de gestionar canales explícitamente.

```
// Configurar e iniciar PWM en un pin
ledcAttach(pin, frequency, resolution_bits);

// Establecer el ciclo de trabajo (duty cycle)
ledcWrite(pin, duty_value);
```

Función	Parámetros	Ejemplo
<code>ledcAttach(pin, freq, res)</code>	Asigna PWM al pin directamente	<code>ledcAttach(5, 8000, 12)</code> → 8kHz, 12 bits (0-4095)
<code>ledcWrite(pin, duty)</code>	Establece ciclo de trabajo	<code>ledcWrite(5, 2048)</code> → 50% duty cycle

Resolución: Por defecto 12 bits (0-4095)

2 Interrupciones y Timers Hardware

Interrupciones Externas (GPIO)

Elemento	Descripción
<code>IRAM_ATTR</code>	Atributo obligatorio en funciones ISR. Guarda la función en RAM interna rápida (IRAM)
<code>attachInterrupt(...)</code>	<code>attachInterrupt(digitalPinToInterrupt(pin), ISR_callback, mode)</code>
Modos disponibles	<code>RISING</code> (flanco ↑) • <code>FALLING</code> (flanco ↓) • <code>CHANGE</code> (cualquier cambio)

Ejemplo de botón con anti-rebote:

```
const int BUTTON_PIN = 4;
volatile int contador = 0;

void IRAM_ATTR handleButtonPress() {
    contador++;
}

void setup() {
    Serial.begin(115200);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    // Configurar interrupción en flanco de bajada
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress,
FALLING);
}

void loop() {
    Serial.printf("Pulsaciones: %d\n", contador);
    delay(500);
}
```

Importante en ISR:

- Mantener código rápido y simple
- No usar `Serial.print()` (es lento)
- Usar variables `volatile` para datos compartidos

Timers Hardware

Los temporizadores por hardware en ESP32 utilizan una API orientada a frecuencia con temporizaciones en microsegundos.

Configuración típica:

```

hw_timer_t *timer = NULL;

// 1. Inicializar a frecuencia de conteo en Hz
timer = timerBegin(timer_frequency);

// 2. Enlazar función ISR
timerAttachInterrupt(timer, &timerInterruptISR);

// 3. Configurar alarma (microsegundos)
timerAlarm(timer, period_us, autoreload, reload_count);

// 4. Control de ejecución
timerStart(timer);
timerStop(timer);

```

Función	Descripción
<code>timerBegin(freq_hz)</code>	Crea timer a frecuencia en Hz. Ej: <code>1000000</code> Hz = 1 tick/μs
<code>timerAttachInterrupt(timer, &ISR)</code>	Enlaza función ISR a ejecutar en cada alarma
<code>timerAlarm(timer, period, reload, count)</code>	Configura alarma: <code>period</code> (μs), <code>reload</code> (boolean periódico), <code>count</code> (0=indefinido)
<code>timerStart(timer)</code>	Inicia la cuenta del timer
<code>timerStop(timer)</code>	Detiene la cuenta del timer

Ejemplo completo - Timer periódico:

```

hw_timer_t *timer = NULL;
volatile int contador = 0;

void IRAM_ATTR onTimerAlarm() {
    contador++;
}

void setup() {
    Serial.begin(115200);

    // Timer a 1 MHz (1 tick = 1 microsegundo)
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimerAlarm);

    // Alarma cada 1,000,000 μs (1 segundo)
    timerAlarm(timer, 1000000, true, 0);
    timerStart(timer);
}

void loop() {
    if (contador > 0) {

```

```

    Serial.printf("Contador: %d\n", contador);
    contador = 0;
    delay(100);
}
}

```

3 Temporizadores por Software (**Ticker.h**)

Ejecuta tareas periódicas de manera asíncrona mediante temporizaciones software.

Función	Descripción	Ejemplo
Ticker ticker;	Declara un objeto Ticker	-
attach(segundos, callback)	Ejecuta callback periódicamente (soporta decimales)	<code>ticker.attach(0.5, myFunction)</code>
attach_ms(milisegundos, callback)	Equivalente en milisegundos	<code>ticker.attach_ms(500, myFunction)</code>
detach()	Detiene la ejecución periódica	<code>ticker.detach()</code>

Ejemplo de uso:

```

#include <Ticker.h>

Ticker blinker;

void blinkLED() {
    digitalWrite(LED_PIN, !digitalRead(LED_PIN));
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    blinker.attach(0.5, blinkLED);
}

void loop() {
    // Ticker funciona en background
}

```

✓ Checklist para ISR (Interrupt Service Routines)

✓ Hacer:

- Operaciones rápidas y simples
- Usar **IRAM_ATTR** obligatoriamente
- Usar variables **volatile**

- Usar `xSemaphoreGiveFromISR()` para señalización

X NO hacer:

- Operaciones lentas (Serial, WiFi)
- Llamadas a `delay()`
- Allocación de memoria
- `printf()` o funciones pesadas

4 Multitarea y FreeRTOS

El ESP32 integra **dos núcleos CPU** (Core 0 y Core 1). FreeRTOS permite gestionar hilos concurrentes en ambos.

Creación y Paginación de Tareas

```
xTaskCreatePinnedToCore(
    TaskFunction,    // void task(void *param)
    "TaskName",      // Etiqueta de depuración
    StackSize,       // Memoria RAM (palabras de 4 bytes)
    Parameters,      // Parámetros entrada (típicamente NULL)
    Priority,         // Prioridad (0=más baja)
    &TaskHandle,     // Manejador de la tarea
    CoreID            // Núcleo (0 o 1)
);
```

Parámetro	Descripción
TaskFunction	Función <code>void nombreTarea(void *pvParameters)</code>
StackSize	Memoria en bytes (ej: <code>2048</code> = 2KB)
Priority	0-24 (mayor valor = mayor prioridad)
CoreID	<code>0</code> = Core 0 • <code>1</code> = Core 1

Funciones Útiles de FreeRTOS

```
// Retardo preciso en tareas periódicas (sin deriva temporal)
vTaskDelayUntil(&xLastWakeTime, xPeriod);

// Obtiene el tick actual del sistema
xLastWakeTime = xTaskGetTickCount();

// Convierte milisegundos a ticks de FreeRTOS
TickType_t xPeriod = pdMS_TO_TICKS(333);
```

```
// Devuelve el núcleo en ejecución (0 o 1)
int coreID = xPortGetCoreID();
```

Función	Propósito	Nota
<code>vTaskDelayUntil()</code>	Retardo preciso periódico	Prefiere a <code>delay()</code> en tareas
<code>xTaskGetTickCount()</code>	Obtiene tick actual	Para sincronización
<code>pdMS_TO_TICKS(ms)</code>	Convierte ms a ticks	Reutilizable
<code>xPortGetCoreID()</code>	Núcleo actual	Útil para debug

Gestión del Ciclo de Vida de Tareas

Función	Descripción
<code>vTaskDelete(NULL)</code>	Elimina la tarea actual. <code>NULL</code> = tarea propia
<code>vTaskDelete(xTaskHandle)</code>	Elimina una tarea específica

Tareas que se ejecutan UNA SOLA VEZ:

```
void vTaskOneShot(void *pvParameters) {
    // Hacer trabajo
    Serial.println("Tarea ejecutada una sola vez");

    // Eliminar la tarea para liberar stack
    vTaskDelete(NULL);
}

void setup() {
    xTaskCreatePinnedToCore(vTaskOneShot, "OneShot", 2048, NULL, 1, NULL, 0);
}

void loop() {
    vTaskDelete(NULL); // Eliminar loop() en FreeRTOS
}
```

Sincronización y Mutex (Protección de Recursos Compartidos)

Conceptos Básicos

Sección Crítica: Bloque de código donde una tarea accede a un recurso compartido.

Mutex: Mecanismo que permite que solo UNA tarea acceda a un recurso a la vez.

Race Condition: Problema que ocurre cuando dos tareas acceden simultáneamente al mismo recurso sin sincronización.

Funciones de Mutex

Función	Descripción
<code>xSemaphoreCreateMutex()</code>	Crea un mutex. Devuelve <code>SemaphoreHandle_t</code> o NULL si falla
<code>xSemaphoreTake(mutex, timeout)</code>	Adquiere el mutex (espera si está ocupado). Timeout: <code>portMAX_DELAY</code> = espera infinita
<code>xSemaphoreGive(mutex)</code>	Libera el mutex para que otras tareas lo usen

Valores de retorno:

- `pdTRUE` (1): Operación exitosa
- `pdFALSE` (0): Timeout alcanzado sin adquirir mutex

✅ Patrón Seguro - Con Mutex

```
#include <Arduino.h>

SemaphoreHandle_t xMutexSerial;

void vTask1(void *pvParameters) {
    for (;;) {
        // Intentar tomar el mutex
        if (xSemaphoreTake(xMutexSerial, portMAX_DELAY) == pdTRUE) {

            // --- SECCIÓN CRÍTICA PROTEGIDA ---
            Serial.println("[TAREA 1] Usando Serial...");
            vTaskDelay(pdMS_TO_TICKS(100));
            Serial.println("[TAREA 1] Finalizó");
            // -----

            // Liberar el mutex
            xSemaphoreGive(xMutexSerial);
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void vTask2(void *pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xMutexSerial, portMAX_DELAY) == pdTRUE) {

            // --- SECCIÓN CRÍTICA PROTEGIDA ---
            Serial.println(" [TAREA 2] Escribiendo...");
            vTaskDelay(pdMS_TO_TICKS(50));
            Serial.println(" [TAREA 2] Hecho");
            // -----

            xSemaphoreGive(xMutexSerial);
        }
    }
}
```



```

        vTaskDelay(pdMS_TO_TICKS(700));
    }
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    // Crear mutex ANTES de las tareas
    xMutexSerial = xSemaphoreCreateMutex();

    if (xMutexSerial != NULL) {
        xTaskCreatePinnedToCore(vTask1, "Task_1", 2048, NULL, 1, NULL, 0);
        xTaskCreatePinnedToCore(vTask2, "Task_2", 2048, NULL, 1, NULL, 1);
    }
}

void loop() {
    vTaskDelete(NULL);
}

```

Ventajas:

-  Escrituras de Serial completas y sin mezcla
-  Acceso ordenado al recurso compartido
-  Evita corrupción de datos

5 Conectividad (WiFi y MQTT)

Gestión de WiFi ([WiFi.h](#))

Función	Descripción	Devuelve
WiFi.begin(SSID, PASSWORD)	Inicializa conexión al Punto de Acceso	-
WiFi.status()	Estado del enlace de red	WL_CONNECTED , WL_DISCONNECTED , etc.
WiFi.localIP()	Dirección IP local asignada	Objeto IPAddress
WiFi.disconnect()	Desconecta de la red WiFi	-
WiFi.RSSI()	Intensidad de señal (dBm)	Negativo, más cercano a 0 = mejor

Ejemplo con reconexión automática:

```

const char *ssid = "MySSID";
const char *password = "MyPassword";
const int TIMEOUT = 10000;

```

```

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    unsigned long start = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - start < TIMEOUT) {
        delay(500);
        Serial.print(".");
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.printf("\nIP: %s | RSSI: %d dBm\n",
            WiFi.localIP().toString().c_str(), WiFi.RSSI());
    } else {
        Serial.println("\nConexion fallida");
    }
}

void loop() {
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println("Reconectando WiFi...");
        WiFi.reconnect();
        delay(5000);
    }
}

```

Protocolo MQTT (PubSubClient.h)

Comunicación cliente/broker bajo patrón **Publicación/Suscripción**.

Inicialización y conexión:

```

WiFiClient espClient;
PubSubClient client(espClient);

// Asignar broker y puerto
client.setServer("broker.example.com", 1883);

// Registrar callback para mensajes recibidos
client.setCallback(OnMqttReceived);

// Conectar con autenticación (opcional)
client.connect("clientId", "user", "password");

// Loop de mantenimiento (llamar en setup() o loop())
client.loop();

```

Publicación y Suscripción:

```

// Publicar mensaje
client.publish("topic/name", "payload_text");

// Suscribirse a tópico
client.subscribe("topic/name");

// Callback para mensajes recibidos
void OnMqttReceived(char *topic, byte *payload, unsigned int length) {
    char message[length + 1];
    memcpy(message, payload, length);
    message[length] = '\0';

    Serial.printf("Topic: %s | Payload: %s\n", topic, message);
}

```

Función	Descripción
<code>setServer(host, port)</code>	Define broker MQTT y puerto
<code>setCallback(function)</code>	Registra función para mensajes recibidos
<code>connect(id, [user], [pass])</code>	Conecta y autentica con broker
<code>loop()</code>	Mantiene conexión y procesa paquetes
<code>publish(topic, payload)</code>	Publica mensaje en tópico
<code>subscribe(topic)</code>	Se suscribe a tópico
<code>connected()</code>	Verifica si está conectado
<code>disconnect()</code>	Desconecta del broker MQTT

Patrón con reconexión automática:

```

#include <PubSubClient.h>
#include <WiFi.h>

WiFiClient espClient;
PubSubClient client(espClient);

void reconnectMQTT() {
    if (client.connected()) return;

    Serial.print("Conectando MQTT...");
    if (client.connect("ESP32", "user", "password")) {
        Serial.println(" OK");
        client.subscribe("sensor/temperatura");
    } else {
        Serial.printf(" Error: %d\n", client.state());
        delay(5000);
    }
}

```

```

}

void loop() {
    if (!client.connected()) {
        reconnectMQTT();
    }
    client.loop();

    static unsigned long lastTime = 0;
    if (millis() - lastTime > 10000) {
        float temp = 25.5;
        client.publish("sensor/temperatura", String(temp).c_str());
        lastTime = millis();
    }
}

```

6 M5Stack Core2: Hardware, Pantalla e IMU

La librería `#include <M5Core2.h>` abstrae el hardware integrado de la placa M5Core2.

Inicialización y Alimentación

Función	Descripción
<code>M5.begin()</code>	Inicializa bus I ² C, pantalla, chip AXP192 y periféricos
<code>M5.update()</code>	Actualiza estado de botones y eventos táctiles (llamar en <code>loop()</code>)
<code>M5.Axp.SetVibration(bool)</code>	Activa (<code>true</code>) o desactiva (<code>false</code>) el motor de vibración

Pantalla LCD TFT

```

// Pintar fondo
M5.Lcd.fillScreen(BLACK);

// Configurar color de texto y fondo
M5.Lcd.setTextColor(WHITE, BLACK);

// Escala del texto (enteros: 1, 2, 3, etc.)
M5.Lcd.setTextSize(2);

// Posicionar cursor en píxeles (x, y)
M5.Lcd.setCursor(10, 20);

// Escribir texto formateado
M5.Lcd.printf("Valor: %d\n", 42);
M5.Lcd.println("Hola Mundo");

```

Función	Descripción
<code>fillScreen(color)</code>	Pinta pantalla del color indicado
<code>setTextColor(text, bg)</code>	Define colores de texto y fondo
<code>setTextSize(size)</code>	Escala del texto
<code>setCursor(x, y)</code>	Posición en píxeles (origen arriba-izquierda)
<code>printf()</code> / <code>println()</code>	Escribe texto formateado

Colores disponibles: BLACK, WHITE, RED, GREEN, BLUE, YELLOW, CYAN, MAGENTA

👉 Pantalla Táctil Capacitiva

```
TouchPoint_t pixel_pos;

if (M5.Touch.ispressed()) {
    pixel_pos = M5.Touch.getPressPoint();
    int16_t x = pixel_pos.x; // Coordenada X (0-319)
    int16_t y = pixel_pos.y; // Coordenada Y (0-239)

    Serial.printf("Touch: (%d, %d)\n", x, y);
}
```

Función	Descripción
<code>M5.Touch.ispressed()</code>	¿Se está tocando la pantalla?
<code>M5.Touch.getPressPoint()</code>	Obtiene coordenadas (x, y) del toque

🌀 Sensor Inercial (IMU 6-Ejes)

Acelerómetro y giroscopio integrados en la placa.

```
// Inicializar IMU
M5.IMU.Init();

// Leer aceleraciones (en G)
float accX, accY, accZ;
M5.IMU.getAccelData(&accX, &accY, &accZ);

// Leer orientación estimada (en grados)
float pitch, roll, yaw;
M5.IMU.getAhrsData(&pitch, &roll, &yaw);

Serial.printf("Acc: (%.2f, %.2f, %.2f) G\n", accX, accY, accZ);
Serial.printf("Ori: P=%.1f° R=%.1f° Y=%.1f°\n", pitch, roll, yaw);
```

Función	Parámetros	Unidades
<code>M5.IMU.Init()</code>	-	Inicialización
<code>getAccelData(&x, &y, &z)</code>	Punteros float	Aceleración en G
<code>getAhrsData(&p, &r, &y)</code>	Punteros float	Ángulos en grados
<code>getGyroData(&x, &y, &z)</code>	Punteros float	Velocidad angular en °/s

Ejes de referencia: X (lateral), Y (frontal), Z (vertical)

Funciones de Utilidad

Mapeo y Restricción de Valores

Función	Descripción	Ejemplo
<code>map(value, fromLow, fromHigh, toLow, toHigh)</code>	Mapea valor de un rango a otro	<code>map(500, 0, 4095, 0, 320)</code>
<code>constrain(value, min, max)</code>	Restringe valor entre límites	<code>constrain(x, 0, 319)</code>
<code>abs(value)</code>	Valor absoluto	<code>abs(-50) → 50</code>
<code>min(a, b)</code>	Mínimo entre dos valores	<code>min(10, 20) → 10</code>
<code>max(a, b)</code>	Máximo entre dos valores	<code>max(10, 20) → 20</code>

Tips, Buenas Prácticas y Patrones

Sincronización y Mutex

- **SIEMPRE crear Mutex ANTES de las tareas** que lo van a usar
- **Secciones críticas cortas:** Minimizar el tiempo dentro de `xSemaphoreTake()` a `xSemaphoreGive()`
- Usar `portMAX_DELAY` cuando se necesite espera infinita en mutex críticos
- **Evitar nesting de mutex:** No tomar mutex A mientras se sostiene mutex B (deadlock)

Optimización y FreeRTOS

- **Prioridad 0-24:** Mayor número = mayor prioridad. Tareas de mayor prioridad preemption tareas bajas
- **Stack size:** Mínimo 2048 bytes para tareas normales. Aumentar si hay uso de buffers o recursión
- `vTaskDelete(NULL)`: Llamar en `loop()` en ESP32/FreeRTOS puro
- `pdMS_TO_TICKS()`: Siempre usar para conversión de ms a ticks (más portátil que hardcoding)
- `vTaskDelayUntil()`: Más preciso que `vTaskDelay()` para tareas periódicas exigentes

Interrupción vs FreeRTOS

- **ISR (`IRAM_ATTR`):** Para eventos críticos y rápidos (GPIO, timers)
- **FreeRTOS Tasks:** Para tareas con ejecuciones de larga duración o control de recursos

Debugging Serial

- **115200 baud:** Estándar. Cambiar solo si hay problemas específicos
- **Serial.printf():** Más flexible que múltiples `Serial.print()`. Usar especificadores: `%d`, `%f`, `%s`
- **Mutex en Serial:** Imprescindible en multi-tarea para mensajes completos
- **Timeout en Serial:** `while (!Serial);` en setup para esperar conexión USB en desarrollo

Patrones Recomendados

Para aplicaciones críticas:

```
// 1. Crear mutex al inicio
xMutex = xSemaphoreCreateMutex();

// 2. Usar en secciones críticas
if (xSemaphoreTake(xMutex, pdMS_TO_TICKS(1000)) == pdTRUE) {
    // Código protegido
    xSemaphoreGive(xMutex);
} else {
    // Timeout: recurso ocupado
    Serial.println("Timeout!");
}
```

Referencias Oficiales

- [Arduino ESP32 Official Documentation](#)
- [FreeRTOS Reference Manual](#)
- [M5Stack Documentation](#)
- [ESP32 Datasheet](#)